

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this course

What we are learning in this session

- ▶ Transactions - TCL Command
- ▶ Error Handling in SQL
- ▶ CTE
- ▶ Temp Table
- ▶ Variables
- ▶ Constraints
- ▶ Clustered vs Nonclustered
- ▶ Trigger
- ▶ Cursor

Let's Start

SQL Commands

SQL Command

- ▶ SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.

1. DDL (Data Definition Language)
2. DML (Data Manipulation Language)
3. DCL (Data Control Language)
4. TCL (Transaction Control Language)
5. DQL(Data Query Language)

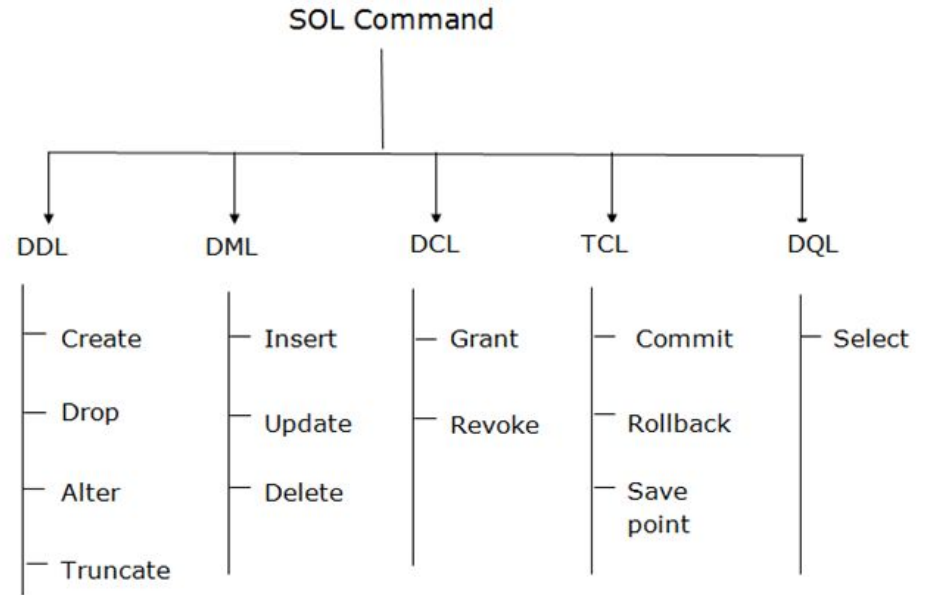
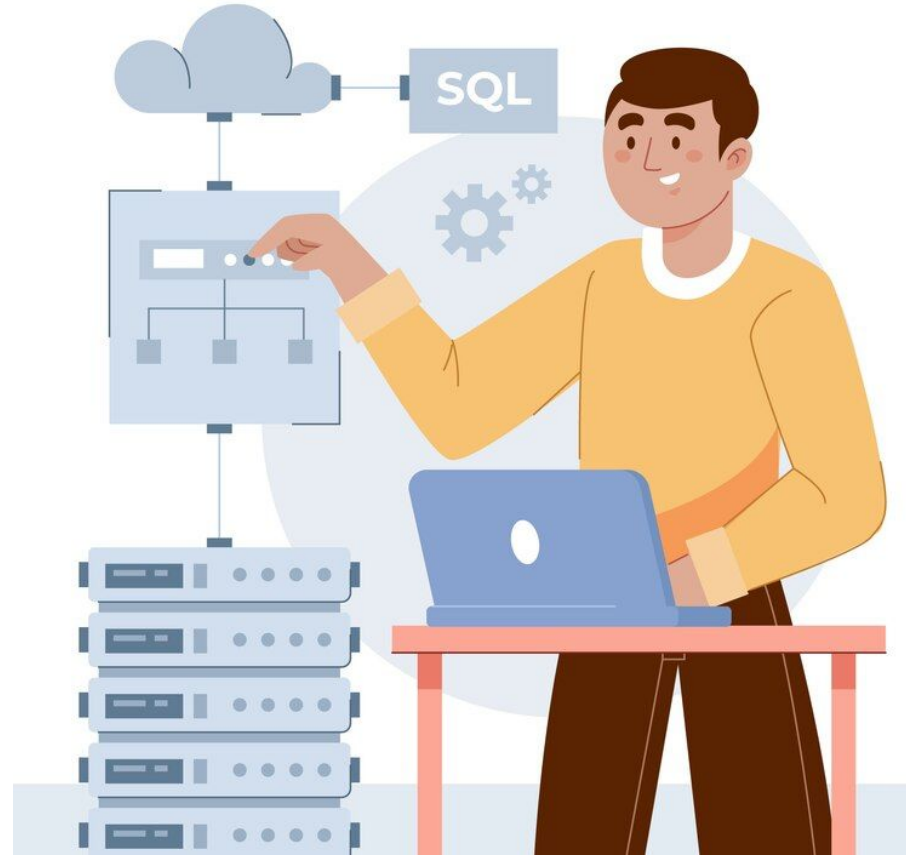


Table - DDL Command



DDL Command

- ▶ In SQL, a table is a structured representation of data organized into rows and columns. It is a fundamental database object used to store, manage, and retrieve information.

- ▶ **Create Table :**

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(50),  
    department VARCHAR(30),  
    salary DECIMAL(10, 2)  
)
```

- ▶ **Alter Table:**

```
ALTER TABLE employees ADD hire_date DATE
```

DDL Command

- ▶ **Drop Table:**

DROP TABLE employees

- ▶ DROP is used to delete database objects, such as tables, views, indexes, or databases themselves.

- ▶ **Truncate Table:**

TRUNCATE TABLE employees

- ▶ TRUNCATE is used to remove all records from a table but retains the table structure for future use. When we use TRUNCATE it will reset column identity.

Syntax:

DELETE FROM your_table WHERE condition
TRUNCATE TABLE your_table

- ▶ DELETE is a row-by-row operation allowing conditional removal, while TRUNCATE efficiently removes all rows in one operation without conditions but may not be suitable if foreign key constraints are present or if you need to log individual deletions.

DQL & DML Command

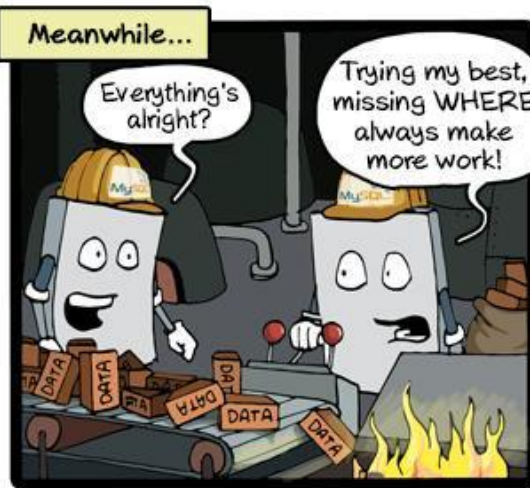
DQL & DML

► DML (Data Manipulation Language) commands in SQL Server are used to interact with and manipulate data stored in tables.

- `SELECT EmployeeID, FirstName, LastName, Salary, DepartmentID
FROM Employee
WHERE DepartmentID = 1`
- `INSERT INTO Employee (FirstName, LastName, Salary, DepartmentID)
VALUES ('John', 'Doe', 50000, 2)`
- `UPDATE Employee
SET Salary = 55000
WHERE EmployeeID = 101`
- `DELETE FROM Employee
WHERE EmployeeID = 102`

DQL & DML

- ORDER BY : Retrieve all employees sorted by their salary in descending order. Default order type is ascending.
 - SELECT * FROM Employee
ORDER BY Salary DESC
- GROUP BY : Count the number of employees in each department
 - SELECT Department, COUNT(*) AS EmployeeCount
FROM Employee
GROUP BY Department
- DISTINCT : Retrieve unique departments from the Employee table
 - SELECT DISTINCT Department
FROM Employee
- SUM, AVG, MIN, MAX
 - SELECT SUM(Salary) AS TotalSalary, AVG(Salary) AS AvgSalary,
MIN(Salary) AS MinSalary, MAX(Salary) AS MaxSalary
FROM Employee



DCL Command

DCL

- ▶ DCL (Data Control Language) commands in SQL are used to control access to the data within a database.
 - GRANT SELECT, INSERT ON Employee TO user1
 - REVOKE DELETE ON Employee FROM user2

TCL Command

TCL

- ▶ TCL (Transaction Control Language) commands in SQL are used to manage transactions.
- ▶
 1. COMMIT: Saves all the changes made during the current transaction to the database.
 2. ROLLBACK: Undoes all changes made during the current transaction, reverting the database to its state before the transaction started.
 3. SAVEPOINT: Sets a named point within a transaction to which you can later roll back.
 4. ROLLBACK TO SAVEPOINT: Rolls back the transaction to a specified savepoint.

```
BEGIN TRANSACTION;
```

```
INSERT INTO Employee (EmployeeID, FirstName, LastName, Salary, DepartmentID) VALUES (101, 'John', 'Doe', 50000, 1);
```

```
UPDATE Employee SET Salary = 55000 WHERE EmployeeID = 101;
```

```
SAVEPOINT my_savepoint;
```

```
INSERT INTO Employee (EmployeeID, FirstName, LastName, Salary, DepartmentID) VALUES (102, 'Jane', 'Smith', 60000, 3);
```

```
ROLLBACK TO SAVEPOINT my_savepoint;
```

```
COMMIT;
```

Error Handling

Error Handling in SQL

- ▶ Error handling in SQL Server gives us control over the Transact-SQL code. For example, when things go wrong, we get a chance to do something about it and possibly make it right again.
- ▶ Why Error Handling?
 - ✓ Prevents unexpected failures
 - ✓ Helps in debugging & troubleshooting
 - ✓ Ensures data consistency
 - ✓ Improves user experience
- ▶ Types of Errors:
 - ✗ **Syntax Errors** – Incorrect SQL syntax
 - ✗ **Runtime Errors** – Division by zero, type mismatches
 - ✗ **Logical Errors** – Incorrect business logic

Error Handling in SQL

▶ Error Handling Methods:

- 1 **TRY...CATCH** – Handles runtime errors
- 2 **RAISEERROR & THROW** – Raises custom errors
- 3 **@@ERROR** – Retrieves error codes

▶ TRY...CATCH Block (Example):

```
BEGIN TRY
    SELECT 1/0 -- Division by zero error
END TRY
BEGIN CATCH
    PRINT 'Error: ' + ERROR_MESSAGE()
END CATCH
```

CTE (Common Table Expression)

CTE

- ▶ A Common Table Expression is a temporary result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement
- ▶ CTEs are particularly useful for making complex queries more readable and maintainable by breaking them down into smaller, logical units. They can also be recursive, allowing you to perform hierarchical queries.
- ▶ WITH SalesCTE (Product, TotalSales) AS (
 SELECT Product, SUM(SalesAmount) AS TotalSales
 FROM Sales
 GROUP BY Product)
SELECT * FROM SalesCTE

CTE

► Benefits :

1. CTEs make code more readable. And readability makes queries easier to debug.
2. CTEs can reference the results multiple times throughout the query. By storing the results of the subquery, you can reuse them throughout a larger query.
3. CTEs can help you perform multi-level aggregations. Use CTEs to store the results of aggregations, which you can then summarize in the main query.

Reference

- <https://stackoverflow.com/questions/54176/what-are-the-advantages-disadvantages-of-using-a-cte>

Temp Table

Temp Table

- ▶ In SQL Server, a temporary table is a table that is created and exists only for the duration of a session or a specific scope of work. Temporary tables are particularly useful when you need to store and manipulate intermediate results within a specific operation, and you don't want to clutter the permanent database with such temporary data.
- ▶ Use Cases for Temporary Tables:
 1. **Complex Queries:** Temporary tables can be used to store intermediate results during complex queries, improving readability and performance.
 2. **Transaction Isolation:** Temporary tables can help isolate data within a transaction, preventing interference with other transactions.
 3. **Reusability:** Temporary tables allow you to reuse the same set of data within a session without having to recreate the table structure.
 4. **Temporary Storage:** Storing data temporarily for a specific process or reporting task.

Temp Table

► Types of Temp Table :

1. Local Temporary Table :

- Local temp tables are similar to Permanent Tables in SQL Server, it accepts the single hash value "#" as the prefix when created. They are visible only to the connection that creates it, and are deleted when the connection is closed.
- ```
CREATE TABLE #TempTable (
 ID INT,
 Name VARCHAR(255)
)
```
- ```
INSERT INTO #TempTable VALUES (1, 'John'), (2, 'Jane')
```
- ```
SELECT * FROM #TempTable
```
- ```
DROP IF EXISTS TABLE #TempTable
```
- ```
SELECT * INTO #TempTable FROM Employee
```

# Temp Table

## ▶ 2. Global Temporary Table

- Global Temporary Tables are also similar to Local Temporary Tables in SQL Server, except two "##" values are used as the prefix at the time of their creation. They are visible to all connections of SQLServer and only destroyed when the last connection referencing the table is closed (in which we have created the Global Temporary Table).
- CREATE TABLE ##GlobalTempTable (  
    ID INT,  
    Name VARCHAR(255)  
)
- INSERT INTO ##GlobalTempTable VALUES (1, 'Mike'), (2, 'Sara')
- SELECT \* FROM ##GlobalTempTable
- DROP IF EXISTS TABLE ##GlobalTempTable
- SELECT \* INTO ##GlobalTempTable FROM Employee

# Variables

# Variable

- ▶ Variable is a storage location with a name that you assign to it. It is used to store and manipulate data temporarily within the scope of a batch, stored procedure, function, or trigger. Variables provide a way to pass data between different parts of a script or stored procedure, making the code more flexible and dynamic.
- ▶ `DECLARE @MyVariable INT`  
`SET @MyVariable = 42`

# Variable

|                  | Temporary Tables                                                                                          | Variables                                                                                                              |
|------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Scope            | Temporary tables are visible across sessions and can be used by multiple queries within the same session. | Variables have a narrower scope and are typically limited to the batch or stored procedure in which they are declared. |
| Data Persistence | Data in temporary tables persists until the session ends or the table is explicitly dropped.              | Data stored in variables only persists for the duration of the batch or procedure execution.                           |
| Performance      | using temporary tables might introduce additional overhead, as they involve disk I/O (writing to tempdb). | variables have lower overhead as they are stored in memory.                                                            |

**Constraints**



# Constraints

- ▶ Constraints in SQL Server are predefined rules and restrictions that are enforced in a single column or multiple columns, regarding the values allowed in the columns, to maintain the integrity, accuracy, and reliability of that column's data.
- ▶ **Types of Constraints:**
  - 1) PRIMARY KEY - Ensures unique, non-null values in a column
  - 2) FOREIGN KEY - Maintains referential integrity between tables
  - 3) UNIQUE - Ensures all values in a column are unique
  - 4) CHECK - Enforces a specific condition on column values
  - 5) DEFAULT - Assigns a default value if none is provided
  - 6) NOT NULL - Ensures a column cannot store NULL values

# Clustered vs Non-Clustered

# Clustered vs Non clustered

## ► What is an Index?

- Improves query performance by organizing data efficiently
- Two main types: **Clustered** and **Non-Clustered**

| Feature          | Clustered Index                       | Non-Clustered Index                        |
|------------------|---------------------------------------|--------------------------------------------|
| Data Storage     | Stores actual data in sorted order    | Stores pointers to actual data             |
| Number per Table | Only 1 per table                      | Multiple allowed                           |
| Speed            | Faster for SELECT queries             | Slower than clustered for large data       |
| Usage            | Ideal for primary keys                | Useful for search columns                  |
| Example          | PRIMARY KEY creates a clustered index | CREATE INDEX creates a non-clustered index |

## ► -- Clustered Index (Automatically created on PK) ►

```
CREATE TABLE Employees (
 EmployeeID INT PRIMARY KEY,
 Name NVARCHAR(50)
);
```

## -- Non-Clustered Index

```
CREATE NONCLUSTERED
INDEX IX_Employees_Name
ON Employees(Name);
```

**Trigger**

# Trigger

- ▶ A trigger is a special type of stored procedure that automatically runs when an event occurs in the database server. DML triggers run when a user tries to modify data through a data manipulation language (DML) event. DML events are INSERT, UPDATE, or DELETE statements on a table or view.
- ▶ DDL triggers run in response to a variety of data definition language (DDL) events. These events primarily correspond to Transact-SQL CREATE, ALTER, and DROP statements
- ▶ Reference  
<https://www.sqlshack.com/triggers-in-sql-server/>  
<https://learn.microsoft.com/en-us/sql/t-sql/statements/create-trigger-transact-sql?view=sql-server-ver16>

# Trigger

## ► Advantages of Triggers

1. **Enforcing Business Rules:** Triggers can be used to enforce business rules by automatically checking and validating data before it is inserted, updated, or deleted. This helps maintain data integrity.
2. **Audit Log:** Triggers can be employed to maintain an audit log of changes, tracking who made the changes and when. This is beneficial for compliance and debugging purposes.
3. **Synchronization of Data:** Triggers can be used to synchronize data across different tables or databases when changes are made, ensuring consistency.

## ► Disadvantages or Drawbacks of Triggers

1. **Performance Impact:** Triggers can introduce performance overhead because they add additional processing during data modification operations.
2. **Hidden Logic:** Triggers contain logic that is not immediately evident when examining the SQL statements for a given operation. This can make it challenging to understand and debug the overall system logic.

**Cursor**

# Cursor

- ▶ Cursor is a database object that allows us to iterate over a result set, row by row, and perform operations on each row individually. Cursors are typically used within Transact-SQL (T-SQL) scripts or stored procedures to process data sequentially.

1. Forward-Only Cursor
2. Static Cursor
3. Dynamic Cursor
4. Keyset-Driven Cursor
5. Scrollable Cursor

- ▶ Reference

<https://learn.microsoft.com/en-us/sql/relational-databases/cursors?view=sql-server-ver16>



# Cursor

## ► Advantages of Cursors

1. **Explicit Control:** Cursors provide explicit control over the fetching, updating, and deleting of rows. This can be useful in situations where we need fine-grained control over data manipulation.
2. **Scrolling:** Cursors provide the ability to move back and forth through the result set, allowing for more flexible navigation compared to a simple query.

## ► Disadvantages of Cursors

1. **Performance Overhead:** Cursors can result in poor performance, especially when processing large result sets, as they may require additional resources and increase database load.
2. **Blocking:** Cursors can lead to increased locking and blocking issues, especially in concurrent environments, where other transactions may be affected by the locks held by the cursor.
3. **Complexity:** Code using cursors tends to be more complex and harder to maintain than set-based operations. SQL is designed to work with sets of data efficiently, and using cursors might deviate from this paradigm.

Thank you